03/03/2021

NUMERICAL METHODS

Ch1) MATHEMATICAL MODELING & PROBLEM SOLVING

Numerical methods are techniques by which mathematical problems are formulated using arithmetic operations.

Non-computer methods:

1) Analytical (Exact) Methods: Used for a limited number of problems

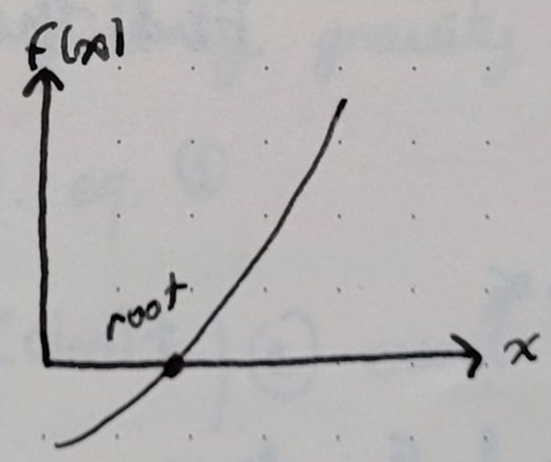
2) Graphical Solutions: Eg. plots. Limited to problems in 3D or fewer dim.

Why study numerical methods?

- NM are extremely powerful problem solving tools.
- Use computer programs
- Learn how to program in MATLAB
  - illustrate the power and limitations of computers.
  - acknowledge and control errors of approximations
- NM reinforce your understanding of mathematics by reducing higher mathematics to basic arithmetic operations

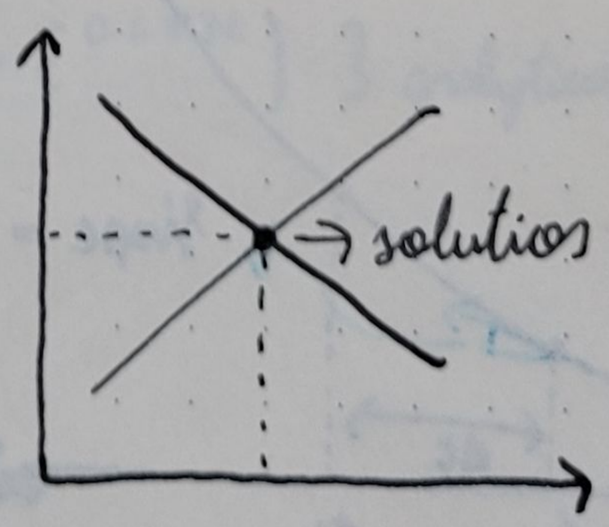
Subject Areas to be Covered

1) Roots of equations
   Solve $f(x)=0$ for x

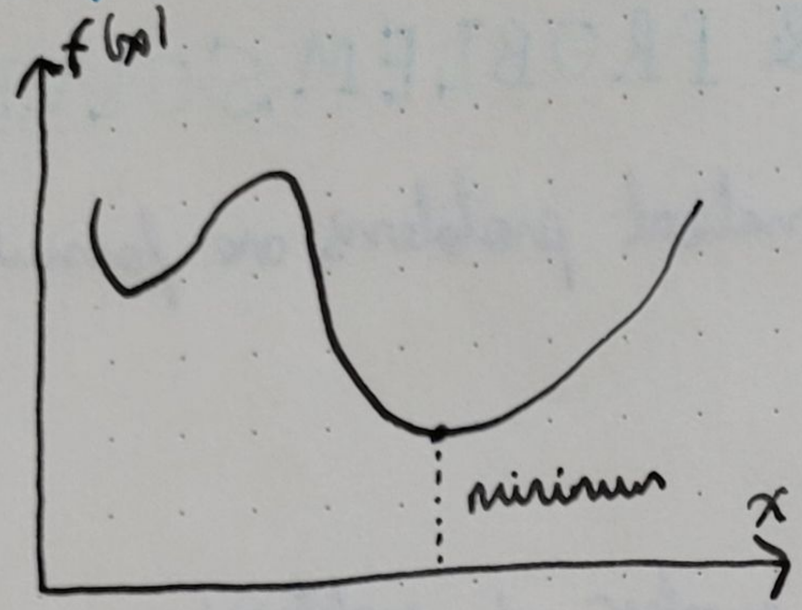


2) Systems of Linear Algebraic Equations

$$\left.\begin{aligned} a_{11}x_1 + a_{12}x_2 &= c_1 \\ a_{21}x_1 + a_{22}x_2 &= c_2 \end{aligned}\right\}$$ Given a's, c's solve x's



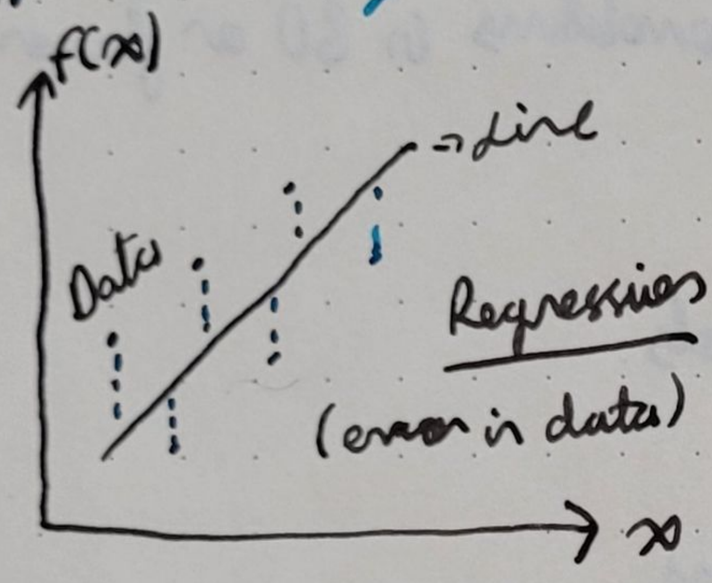
3) Optimization

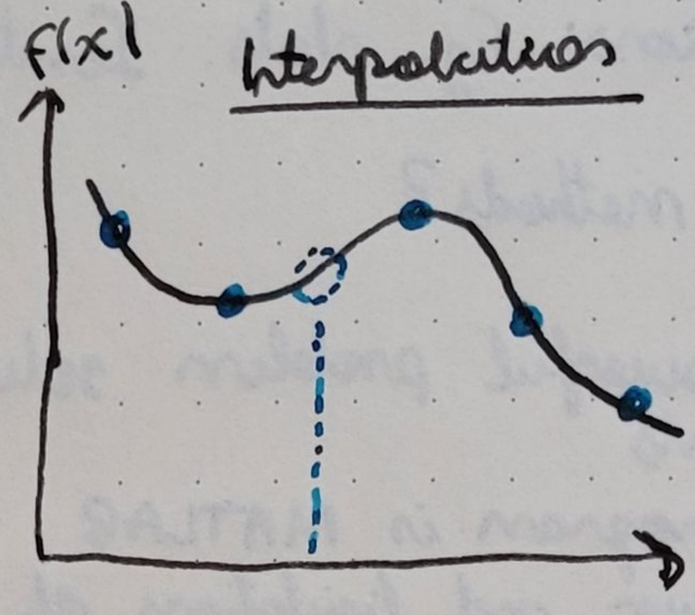


Determine x that gives optimum f(x)

- single and multi-variable unconstrained optimization
- constrained optimization (linear programming)

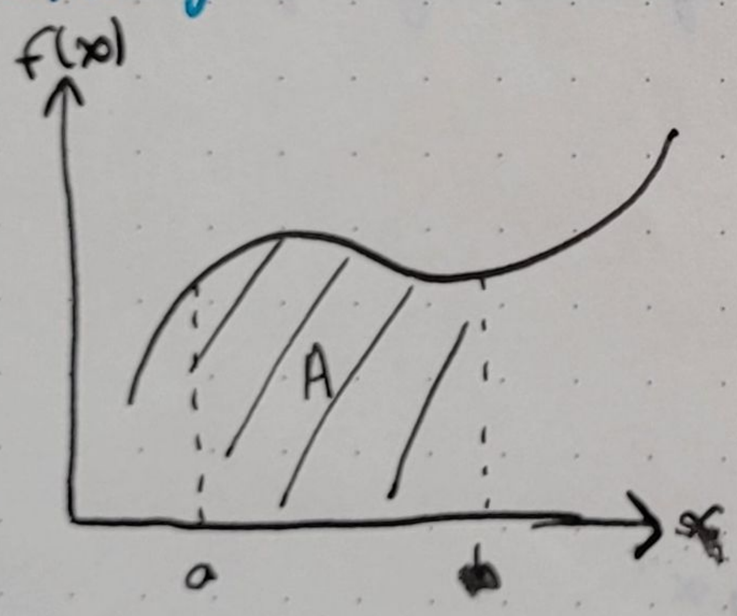
4) Curve Fitting





determine intermediate values for error-free data

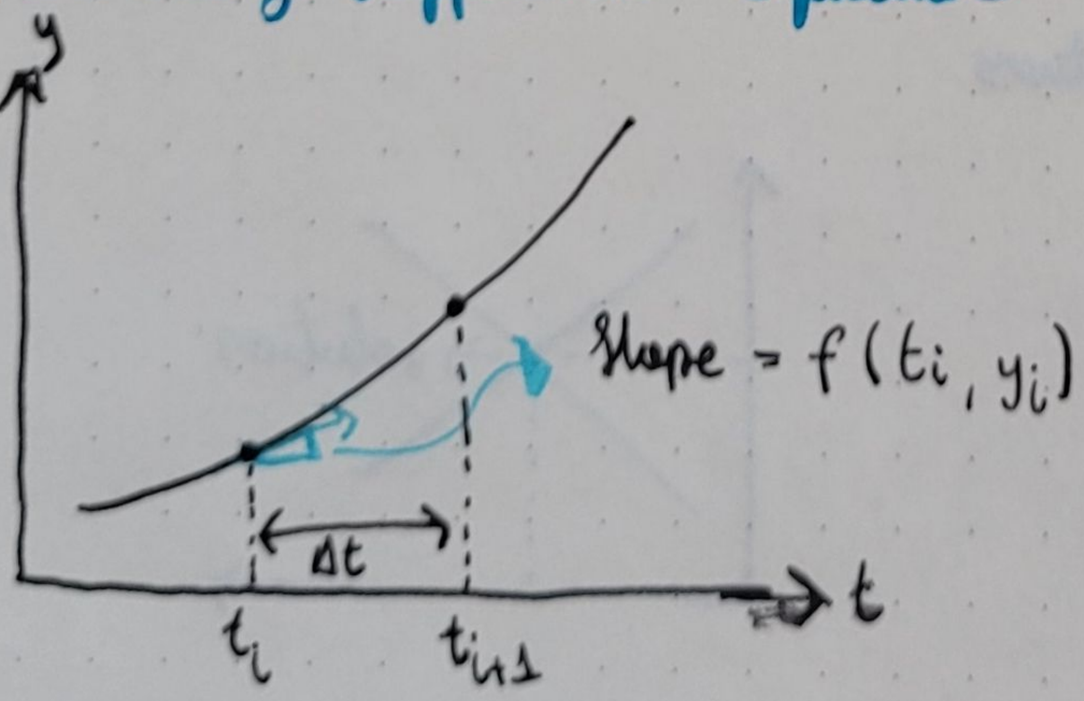
5) Integration



$$A = \int_a^b f(x)\,dx$$

Find the area under the curve

6) Ordinary Differential Equations



Gives $\frac{dy}{dt} \cong \frac{\Delta y}{\Delta t} = f(t, y)$

Solve for y as a function of t

$$y_{i+1} = y_i + f(t_i, y_i)\,\Delta t$$

04/03/2023

Mathematical Model

A formulation or equation that expresses essential features of a physical system or process in mathematical terms.

Dependent variable = f (independent var. → generally time or space
parameters
forcing functions → external influences)

↓
reflects the state or behavior of the system

Ex Newton's second law of motion $F = m \cdot a$

$a = \frac{F}{m}$ → forcing functions
↳ parameter (no independent var.)

↓
dependent var.

↳ acceleration (m/s^2)
↳ mass (kg)
↳ net force acting on a body ($N, \frac{kgm}{s^2}$)

• Use Newton's second law to determine the terminal velocity of a free falling body.

$\frac{dv}{dt} = \frac{F}{m}$

F_u: upward pull by air resistance $F_u = -cv$

F_d: downward pull by gravity $F_d = m \cdot g$

$\frac{dv}{dt} = \frac{mg - cv}{m} = g - \frac{c}{m} v$ } a diff. eq. ①

analytical sol.: $v(t) = \frac{gm}{c}\left(1 - e^{-(c/m)t}\right)$ ② exact solution.

↙ dep var. ↘ indep. var.

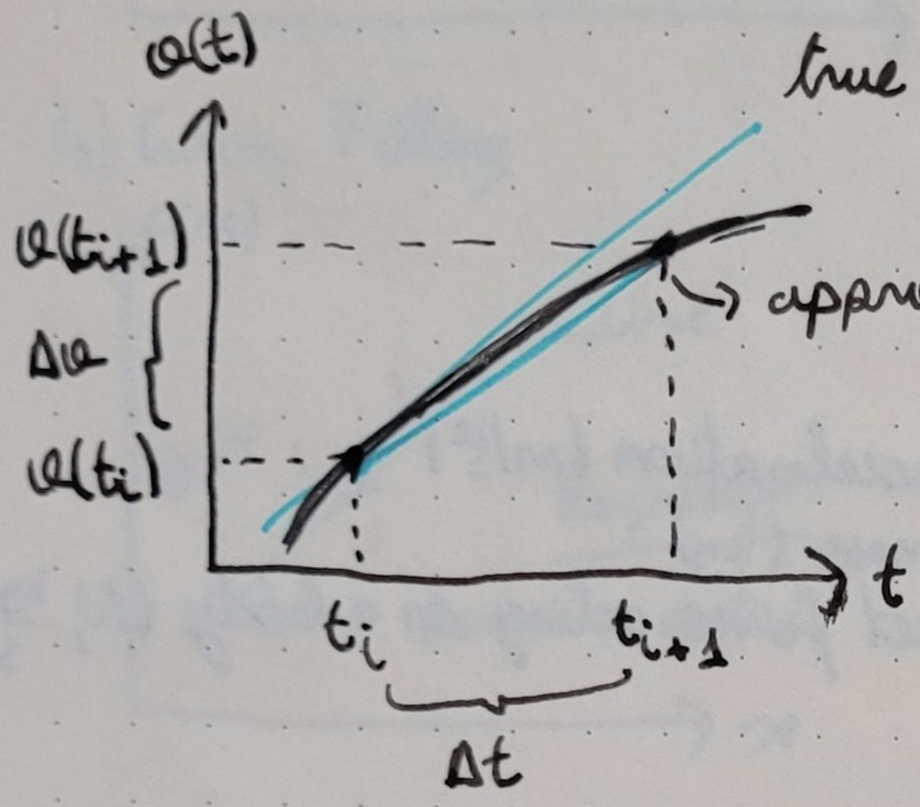
parachutist: $m = 68.1\,kg$ $v(t) = 53.44\left(1 - e^{-0.193t}\right)$ } analytical solution
$c = 12.5\,kg/s$
$g = 9.81\,m/s^2$

t	v
0	0
2	16.42
10	44.92
⋮	
∞	53.44 → $F_u = F_d$ → $F_{net} = 0$

Numerical method: Reformulate the problem so that it can be solved by arithmetic operations

For Newton's second law: $\frac{dv}{dt} \cong \frac{\Delta v}{\Delta t} = \frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i}$ (3)

$\frac{dv}{dt} = \lim_{\Delta t \to 0} \frac{\Delta v}{\Delta t}$ $v(t_i)$: initial velocity $v(t_{i+1})$: velocity at some later time



(3) finite divided difference approximation of the derivatives at time ti

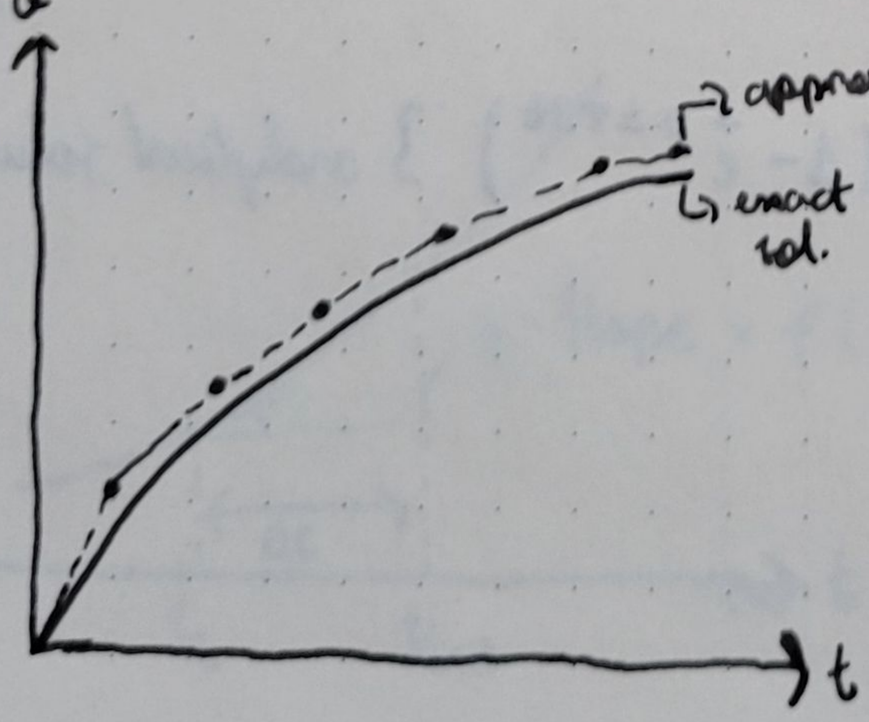
$$\frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i} = g - \frac{c}{m} v(t_i) \quad \} \text{ slope approximate}$$

Numerical solution: $v(t_{i+1}) = v(t_i) + \underbrace{\left[g - \frac{c}{m} v(t_i)\right]}_{\text{slope}} \underbrace{(t_{i+1} - t_i)}_{\text{step size}}$ (4)

Falling parachutist: $m = 68.1$ kg, $c = 12.5$ kg/sec, $\Delta t = 2$ s

at t_i $v = 0$

at t_{i+1} $v = 0 + \left[9.81 - \frac{12.5}{68.1} \cdot 0\right] \cdot 2 = 19.62$ m/s



- Can minimize differences by using smaller step size
- There's a tradeoff between accuracy and computational effort.

18/03/21

CHAPTER 3

Approximations and Round-off Errors

We want to limit errors in our numerical solutions, since they involve approximat.

- Numerical errors

Roundoff error: Computers can only represent numbers with a finite number of digits.

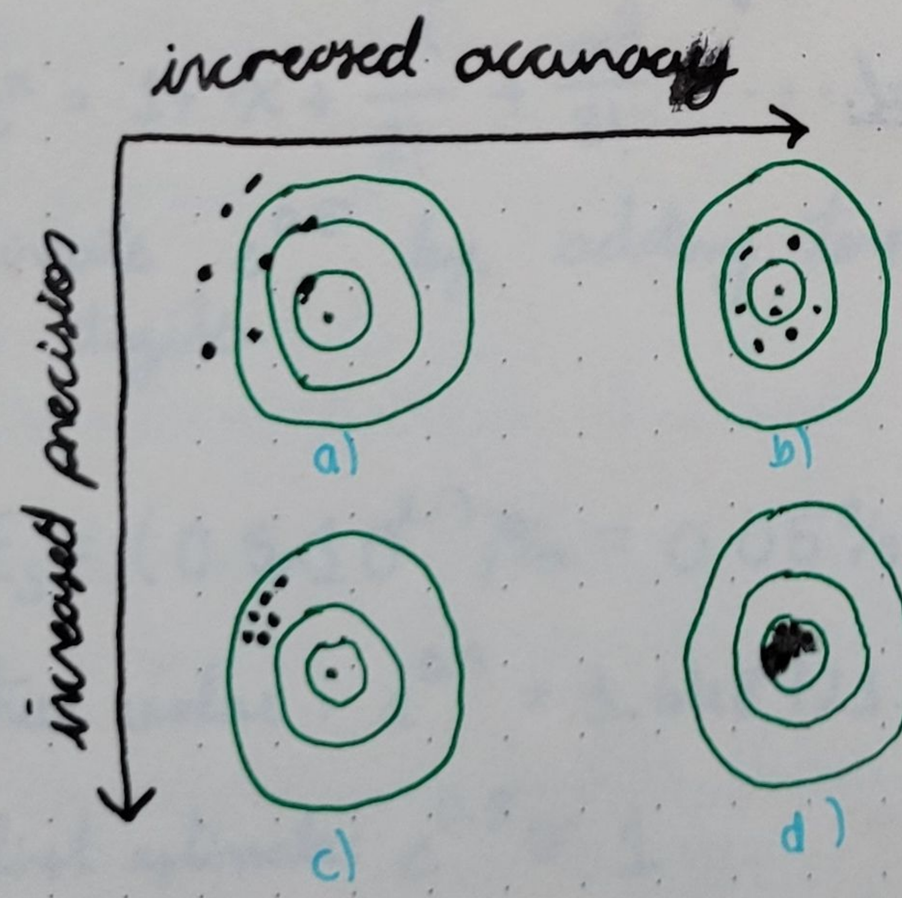
Truncation error: Introduced by the numerical methods which use approximations to represent exact mathematical operations.

- Significant Figures

Significant digits of a number are those that can be used with confidence. They correspond to the # of certain digits plus one estimated digit.

$4.53 \times 10^4 \rightarrow$ 3 significant
$4.530 \times 10^4 \rightarrow$ 4 significant

Accuracy: How closely a computed/measured value agrees with the true value.
Precision: How closely individual computed/measured values agree with each other.



a) inaccurate and imprecise
b) accurate and imprecise
c) inaccurate and precise
d) accurate and precise

- Errors
Represents both the inaccuracy and imprecise

Truncation error: result when approximations are used to represent exact mathematical procedures.

Round of errors when numbers having limited significant figures are used to represent exact numbers.

$$\text{True value} = \text{approximation} + \text{error}$$

- true error E_t = true value − approximation

We can normalize the error using the true value to account for the magnitudes of quantities.

$$\text{True fractional relative error} = \frac{\text{true error}}{\text{true value}}$$

- true percent relative error -

$$\varepsilon_t = \frac{\text{true error}}{\text{true value}} \times 100\%$$

Ex. Tasks of measuring the lengths of a bridge and a rivet

	true val	measured val.
bridge	10,000cm	9999cm
rivet	10cm	9cm

a) $E_t = 10000 - 9999 = 1$ (bridge)

$E_t = 10 - 9 = 1$ (rivet)

b) $\varepsilon_t = \frac{1}{10000} = 0,01\%$

$\varepsilon_t = \frac{1}{10} = 1\%$

In real life true value is unknown. Alternative method, normalize the error using the best available estimate of the true value.

$$\varepsilon_a = \frac{\text{approximate error}}{\text{approximation}} \cdot 100\%$$

In <u>iterative approaches</u> successively compute better approximations.

percent relative error:

$$\varepsilon_a = \frac{\text{curr. approximation} - \text{prev. approximation}}{\text{curr. approx.}} \cdot 100\%$$

repeat iterations until:

$|\varepsilon_a| < \varepsilon_s$ → prespecified percent tolerance

It can be shown that if $\varepsilon_s = (0.5 \times 10^{2-n})\%$ the result is correct to at least n <u>significant</u> digits.

Ex Error estimates for iterative methods

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!} + \dots \quad \text{(Maclaurin series expansion)}$$

Estimate $e^{0.5}$ by adding terms until ε_a is less than ε_s, conforming to 3 sig. digits.

Soln

$\varepsilon_s = (0.5 \times 10^{2-3})\% = 0.05\%$

true value: $e^{0.5} = 1.648721\dots$

first estimate: $e^{0.5} \cong 1$

$$\varepsilon_t = \frac{1.648721 - 1.5}{1.648721} \cdot 100\% = 9.02\%$$

$$\varepsilon_a = \frac{1.5 - 1}{1.5} = 33.3\% \quad \text{(not less than } \varepsilon_s\text{)}$$

second estimate: $e^{0.5} \cong 1 + 0.5 = 1.5$

⋮

6th estimate: $e^{0.5} \cong 1.648697 \rightarrow \varepsilon_t = 0.00142\%$

$\varepsilon_a = 0.0158\% < \varepsilon_s = 0.05\%$

Round-off Errors

Numbers such as π, e, $\sqrt{83}$ can't be represented exactly by a computer.

Computer representation of numbers:

binary digits = Bits words = consist of bits

- Number systems

Signed Magnitude Method

First bit for the sign (0+, 1-)
Remaining bits for the number

1 0000000 1010 1101 } word 16 bits
-173

Range of Integers

Determine range of integers in base-10 that can be represented on a 16 bit computer.

First bit holds sign the other 15 bits from 0 to $\overbrace{111..1}^{15}$ upper limit

$(1\times2^{14}) + (1\times2^{13}) + \dots + (1\times2^{0}) = 32767 = 2^{15}-1$

Range is -32767 to 32767 it is redundant to use minus zero (1000...0)
It is employed to represent an additional negative number

-32768 to 32767

Using 32 bits, largest integer is +2147483647

Fixed-point versus Floating Point Representation

In engineering normally at least 7 to 8 significant digits precision is required
Hence number of bits to encode 8 decimal digits is approx. 26
(since $\log_2 10 = 3{,}32$ bits are needed on the average to encode a digit)

Bits: 31 | 30 ... 8 | 7 ... 0 } with this fixed-pt rep. range is small
S | integer part | fraction

Largest: 0|1111....1 = 16777215.998046875

Smallest: 0|000.....1 = 0.00390625

- floating point

32 bits are divided to two parts:

Mantisse with its sign (represents fractions)
Exponent with its sign (power of 2 which multiplies mantissa)

$$\underbrace{(\text{sign})}_{\text{1 bit}} \times \underbrace{\text{mantissa}}_{\substack{\text{binary frac} \\ \text{with a nonzero} \\ \text{leading bit}}} \times 2^{\pm \underbrace{\text{exponent}}_{\text{binary integer}}}$$

Q) How many bits are used to the mantisse?

A) To achieve at least 7 significant decimal digits approx 23 bits

Q) How many bits are used for the exponent?

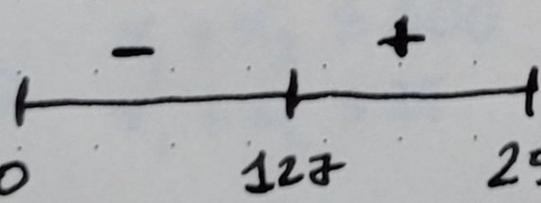
A) 8 bits

Q) How to represent sign of the exponent?

A) Exponent may be represented in sign magnitude form ⇒ range [-127, 127]

Disadvantage: two rep. for zero (+0, -0). To avoid this one may use biased format to represent the exponent, where the exponent has no sign

The range 0 to 255 of an 8-bit number is divided into two parts: 0-127 and 128-255



} range of the exponent -127 → +128

The largest floating-pt number using 32 bits:

$$0.111\ldots1 \times 2^{11111111} \cong 3.4 \times 10^{38}$$

The smallest:

$$0.1\ldots\ldots0 \times 2^{-127} \cong 0.293 \times 10^{-38}$$

IEEE754 standard uses the most significant bit of mantisse as always 1, on the left of the -virtual- decimal point.

$$(-1)^S \times (1.f)_2 \times 2^{exp-127}$$